

PayNaija Data Platform

Business scenarios and requirements, Project 1 through Project 8

PayNaija is a fictional fintech operating across Nigeria, Ghana, and Kenya, used as the business backdrop for a structured Apache Airflow mentorship. Each project below was handed off the way a product manager would actually brief a data engineer: a business scenario, then a list of concrete requirements, with no implementation details specified in advance. The design and the code that answered each of these requirements live in the accompanying GitHub repository.

Project 1: Basic Signups ETL

Airflow concepts introduced: DAGs, TaskFlow API, task dependencies

Business scenario

PayNaija's product team exports a daily `signups.csv` from the internal admin tool. An analyst manually cleans this file in Excel every morning and pastes it into a reporting spreadsheet for the BI team. It is slow, error prone, and does not scale. The pipeline needs to be automated so the data lands in Postgres every day without anyone touching it.

Business requirements

- 1 The CSV contains `signup_id`, `full_name`, `email`, `phone_number`, `signup_date`, `referral_source`, and `plan_type`.
- 2 Some rows will have missing or malformed data. Bad data must not silently corrupt the reporting table.
- 3 Clean data must land in a Postgres table called `customer_signups` that the BI team can query directly.
- 4 This is a first version. Scheduling and retries are not required yet, just one correct manual run end to end.

Project 2: Reliability: Scheduling, Retries, and Logging

Airflow concepts introduced: Scheduling, retries, retry_delay, structured logging

Business scenario

The manual-run version works, but the ops team needs this running every single day without a human triggering it. A brief network blip during testing killed the whole pipeline outright, which is not acceptable, and the logs from that failure were unreadable, just scattered print statements with no way to tell what actually broke.

Business requirements

- 1 The DAG must run automatically once per day with no manual triggering.
- 2 Transient failures, such as a brief database connection drop, should not immediately fail the whole pipeline.
- 3 Not every failure deserves a retry. Deterministic failures should fail fast rather than retry pointlessly.
- 4 Whoever is on call needs enough information in the logs to understand what failed and why, without opening the source code.

Project 3: Exchange Rate Enrichment Pipeline

Airflow concepts introduced: API calls in tasks, raw and cleaned storage (bronze/silver), XCom sizing

Business scenario

PayNaija operates across several African countries and needs to report transaction values in a common currency. Someone currently checks a public exchange rate site every morning and types the numbers into a spreadsheet. This needs to be automated, pulling from a public API.

Business requirements

- 1 Call a public exchange rate API once per day for the currencies relevant to PayNaija (NGN, GHS, KES against USD).
- 2 The raw JSON response must be preserved exactly as received, for audit purposes, indefinitely.
- 3 A cleaned, tabular version must also exist, ready for reporting.
- 4 The pipeline must handle the reality that a third-party API is less predictable than a static file.
- 5 This is a manual-run-first version. Retries and logging polish are layered on afterward.

Project 4: Waiting for the Bank Settlement File

Airflow concepts introduced: FileSensor, poke_interval, timeout, mode, soft_fail

Business scenario

A daily bank settlement file is dropped by a separate Treasury Ops team onto a shared server sometime in the morning, historically anywhere from 6am to past 11am, and occasionally not at all. The pipeline cannot fail the instant the file is not there at 6:01am, but it also cannot wait forever.

Business requirements

- 1 The DAG should start looking for the settlement file early in the morning.
- 2 It should check periodically rather than continuously, and rather than failing on the first missed check.
- 3 There must be a reasonable cutoff. If the file has not appeared by early-to-mid afternoon, the DAG should fail loudly.
- 4 Once the file is confirmed present, hand off to downstream processing.
- 5 On-call staff must be able to tell, from the logs, whether the failure was a missing file or something else entirely.

Project 5: Configuration: Airflow Variables and Jinja Templating

Airflow concepts introduced: Airflow Variables, Variable.get, Jinja templating, template_fields

Business scenario

The DAGs work, but file paths, currency lists, and table names are hardcoded throughout the source. Every time something changes, such as finance wanting to add a new currency, someone has to open the DAG source, edit Python, and redeploy. File paths are also fixed single strings, even though a file that lands daily should reflect the actual date it lands on.

Business requirements

- 1 Move values that operations or finance might reasonably need to change into Airflow Variables, editable from the UI.
- 2 Introduce Jinja templating so file paths reflect the actual logical date of a run, instead of a single fixed filename.
- 3 Be deliberate about what belongs in a Variable versus what stays a Python constant. Not everything benefits from being externalized.

Project 6: Branching on Business Logic

Airflow concepts introduced: BranchPythonOperator, @task.branch, trigger rules for reconvergence

Business scenario

Finance flagged a bad GHS exchange rate that looked wildly off from the previous day, roughly a 40 percent overnight jump. It turned out to be a bad data point from the provider, and it went straight into the reporting table before anyone caught it. A human should not have to check this every day, but an obviously broken rate should not flow into the reporting table unsupervised either.

Business requirements

- 1 After the rates are cleaned, the pipeline must decide, per run, whether the data looks normal or anomalous.
- 2 Anomalous means any currency's new rate differs from the previous loaded rate by more than roughly 15 percent.
- 3 If the data looks normal, proceed exactly as before and load straight into the reporting table.
- 4 If the data looks anomalous, do not load into production. Route it somewhere for manual review instead.
- 5 This must be a runtime decision made by the DAG itself, not something a human triggers by hand each day.

Project 7: Cross-DAG Triggering and Notifications

Airflow concepts introduced: TriggerDagRunOperator, on_failure_callback, distinguishing business alerts from operational alerts

Business scenario

When a rate gets flagged as anomalous, nobody should have to remember to check the flagged table manually. Separately, the settlement pipeline and the exchange rate pipeline both feed into a weekly finance reconciliation report, and nobody should have to babysit two DAGs and manually kick off reconciliation once both are done.

Business requirements

- 1 When an anomaly is detected, send an email notification immediately as part of that DAG run.
- 2 If the exchange rate pipeline or the settlement pipeline fails entirely, someone should be notified, separately from an anomaly alert.
- 3 Build a new, minimal reconciliation DAG for now. The point is the triggering mechanism, not new business logic yet.
- 4 Once the settlement pipeline finishes successfully, it should automatically trigger the reconciliation DAG with no manual intervention.

Project 8: The Capstone: Weekly Reconciliation, For Real

Airflow concepts introduced: Cross-DAG data passing via conf and logical_date, idempotent aggregation, resilience to partial system failure

Business scenario

Every week, finance needs a reconciliation summary pulling together total signups, the exchange rates used, and confirmation that the settlement file was processed, into one report. It cannot be fragile. If FX rates were flagged as anomalous that week, the report must show that, not hide it. If a settlement file never arrived on a given day, the report must say so honestly, not pretend everything was fine.

Business requirements

- 1 The reconciliation DAG needs real logic: query signups, clean exchange rates, and flagged exchange rates for the relevant week, and produce a genuine summary artifact.
- 2 The report must honestly reflect gaps. A missing settlement day or a flagged rate must be visible, never silently omitted.
- 3 The reconciliation DAG is triggered by a daily pipeline but represents a weekly summary. This mismatch needs a deliberate architectural answer.
- 4 The whole system must be resilient to realistic partial failure: an anomaly gets flagged, a settlement file never lands, an API call fails and retries. None of this should silently corrupt or crash the final report.